

Search Under Disorder and Query Entropy

A two-budget note on repeated comparison search

Jonathan R. Landers October 8, 2026

The disorder potential in *Disorder Potential and the Cost of Search* limits how far entries can drift from their sorted ranks. It says nothing about which targets will be requested. Even a sorted array needs logarithmic worst-case search when every key or gap is possible. This note gives the targets a known distribution by rank. The resulting bound separates two costs: disorder controls the comparisons needed to build a sorted index, while target uncertainty controls the expected comparisons after it exists.

Array promise and query prior. Let $A = (a_1, \dots, a_n)$ contain distinct real keys, and let x_i be the sorted rank of a_i . As in the paper, assume the supplied promise

$$V(A) = \frac{1}{2} \sum_{i=1}^n (x_i - i)^2 \leq E, \quad D_E = \min \left\{ n - 1, \left\lfloor \frac{\sqrt{1 + 8E} - 1}{2} \right\rfloor \right\}, \quad R_E = D_E + 1.$$

The displacement lemma gives $|x_i - i| \leq D_E$: if one entry moves by d , the other displacements sum to $-d$, so $2V(A) \geq d(d + 1)$. Without preprocessing, one target costs $O(\log_2(n + 1) + R_E)$ comparisons in the worst case.

A target lies in one of the $2n + 1$ ordered cells

$$g_0, k_1, g_1, k_2, \dots, k_n, g_n,$$

where k_j is the key of rank j and g_j is the gap after it. Let w be known probabilities on these cells, with $H(w) = -\sum_c w_c \log_2 w_c$ and $0 \log_2 0 = 0$. Conditional on the fixed array, targets are independent draws with this cell law. Their values within a gap do not affect comparisons. The law concerns *target ranks*, not the numerical distribution of array values.

Preprocessing may compare array keys and retain an $O(n)$ -word index. During a query, only target-to-array-key comparisons reveal order; they are charged, and the algorithm must answer correctly for every real target, including cells of probability zero. Define $\mathcal{T}_q(n, E, w)$ as the minimum, over these algorithms, of the worst-case over $V(A) \leq E$ of preprocessing comparisons plus expected comparisons for q targets.

Theorem 1 (Two-budget search bound). *For $n \geq 2$ and $q \geq 1$, with an absolute constant C ,*

$$\boxed{\frac{qH(w)}{\log_2 3} \leq \mathcal{T}_q(n, E, w) \leq C \min \{q(\log_2(n + 1) + R_E), n \log_2(R_E + 1) + q(H(w) + 2)\}}. \quad (1)$$

When $E = 0$, the sorted-index preprocessing in the second term uses zero comparisons.

If w is spread roughly uniformly, $H(w) = \Theta(\log n)$ and the new term gives no asymptotic saving per target. If queries are supported on m cells, then $H(w) \leq \log_2 m$; after the index is built, their expected search cost is $O(\log m + 1)$, even though the algorithm remains correct on every target. The index cost is paid once and can be amortized across many queries.

The saving does not require impossible queries. Suppose $1 - \varepsilon$ of the mass is uniform on m favored key cells and ε is uniform on all other cells. Then every cell has positive probability, yet

$$H(w) = h_2(\varepsilon) + (1 - \varepsilon) \log_2 m + \varepsilon \log_2(2n + 1 - m),$$

where h_2 is binary entropy. With fixed m and $\varepsilon = 1/\log_2 n$, this is $O(1)$: after indexing, the expected comparisons per target are constant, while rare targets still receive correct answers.

Proof of the upper bound. The first choice in (1) repeats the potential-based search: midpoint comparisons shrink the candidate interval up to a margin $2D_E$, followed by a scan of $O(R_E)$ positions.

For the second choice, build a sorted index of the array positions. Since the entry of rank j occurs by position $j + D_E$, begin with a min-heap containing positions $1, \dots, \min\{n, R_E\}$. Extract its minimum, insert the next unread position, and repeat. Before the j th extraction, the heap contains every unread position through $\min\{n, j + D_E\}$; in particular it contains rank j . Each extraction therefore yields the next key in sorted order. The heap has at most R_E entries, so the index costs $O(n \log_2(R_E + 1))$ array-key comparisons and $O(n)$ words. For $D_E = 0$, the array itself is already the index.

Now search this index with an ordered decision tree chosen for w . For every cell c with $w_c > 0$, place a dyadic interval of length $2^{-\ell_c}$ inside its cumulative-probability interval, where $\ell_c = \lceil \log_2(1/w_c) \rceil + 1$. These intervals give an alphabetic prefix code: their left-to-right order is the cell order, and their mean depth is less than $H(w) + 2$. Zero-weight cells can be included by an arbitrarily small perturbation. Every split between consecutive cells is a threshold at some indexed key: a cut just before or after rank j uses $y < k_j$ or $y \leq k_j$, respectively. One target-to-key comparison therefore implements the binary branch. Thus the mean search cost is $O(H(w) + 2)$, proving the second choice. This is the classical entropy-sensitive search-tree construction.¹

Why the entropy term is necessary. Fix any array and any preprocessing transcript. A comparison of the target with an array key has at most three outcomes. A correct search tree must distinguish its possible key and gap cells: an absent leaf cannot cover gaps on both sides of a key without also admitting that key. The leaf depths therefore form a ternary prefix code for w , whose expected depth is at least $H(w)/\log_2 3$. Independence makes the same conditional bound hold after any history of earlier queries. Summing over q proves the left side of (1).

When the saving pays. Writing $B_E = O(n \log_2(R_E + 1))$, the two strategies give an average cost per target of at most

$$O\left(\min\left\{\log_2(n+1) + R_E, \frac{B_E}{q} + H(w) + 1\right\}\right). \quad (2)$$

For $E = 0$, $B_E = 0$ and a known query prior helps immediately. For a disordered array, enough queries must share the index to repay its construction. This is an amortized gain in expected search depth, not a claim that the worst-case target has become easier.

What the bound changes. The original $\Omega(\log n)$ obstruction concerns a worst-case target; it remains when all cells are possible and no query prior is used. A law for the *array's* potential, beta-shaped or otherwise, can reduce the displacement work but does not lower target entropy. Here the search-depth saving is an expectation over a stated query law, and the sorted index that makes it possible is charged explicitly. A sharper no-preprocessing result of the form $O(H(w) + R_E)$ per query would require a separate search argument; (1) makes no such claim.

Takeaway. Disorder sets the one-time price of learning the array's order, and query entropy sets the average price of using that order. When targets concentrate on a few ranks and enough queries share the index, the recurring $\log n$ search term becomes $H(w) + O(1)$ without giving up correct answers for rare targets.

Context. The potential, displacement lemma, and single-search theorem are from Landers, *Disorder Potential and the Cost of Search*, version 2. The heap-based index construction comes from Landers, *Search Under a Disorder Budget*.

¹K. Mehlhorn, *Nearly Optimal Binary Search Trees*, Acta Informatica 5 (1975), 287–295. Author-hosted paper.